

The Drink Shaker

A party drink app that only ever suggests drinks you can actually pour. This document is everything another engineer or coding agent needs to work on it.

Live site	https://drinkapp-rust.vercel.app
Repository	https://github.com/todrod/drinkapp (branch: main)
Local path	C:\Users\todro\Desktop\projects\drinking app\shaker
Architecture spec	docs/SPEC.html in the repo
Stack	Expo SDK 57 - React Native 0.86 - TypeScript - React 19
Deployment	Vercel, automatic on push to main
Bundle size	~1.15 MB web export

1. What it does

Tab	Purpose
Random	Spin - or physically shake the phone - for a drink makeable right now
Cabinet	Inventory: add, remove, track remaining quantity
Recipes	96-recipe library with search, filters, ratings, favourites
Builder	Pick base, mixers and toppings from stock; see what it matches

The repo root **is** the Expo project - there is no nesting. With the owner's 116-bottle bar loaded, 81 of the 96 recipes are pourable.

2. Running it

```
npm install
npm start          # Expo dev server; scan QR with Expo Go
npm run web        # browser instead
npm run typecheck  # tsc --noEmit (must pass before commit)
npm run build:web  # static export + PWA injection -> dist/
```

Vercel builds on every push to **main** using vercel.json. No dashboard configuration is required.

3. Where the important code lives

Path	What
src/logic/generator.ts	The matching algorithm. Start here.
src/store.tsx	Context provider - kit, recipes, prefs, all mutators
src/db/store.ts + store.web.ts	Persistence: SQLite on native, localStorage on web

src/data/catalog.ts + bar.ts	~147 catalog items; MY_BAR is the owner's real shelf
src/data/recipes.ts + library.ts	96 recipes: 59 core, 12 from books, 24 themed
src/data/liquids.ts	Canonical liquid palette; derives each card's colour
src/theme.ts	Design tokens and the cross-platform SHADOW helper
src/api/trending.ts	Deferred feature - types only, no network
scripts/	Icon-font subsetting, PWA post-build, asset pipeline

4. The data model

Three tables and one idea. **CatalogItem** is the bundled library of known ingredients with stable ids (sp-gin, mx-lime-juice). **InventoryItem** is what the user owns and points at a catalog item. **DrinkRecipe** holds ingredients that *also* point at catalog ids.

The key idea: a recipe ingredient references a catalog id, not a string. So "can I pour this?" is a set lookup rather than fuzzy name matching.

5. The matching algorithm

An ingredient resolves if the user owns its catalog id, or it is a staple (ice, salt), or they typed the name freehand, or - only when substitutions are enabled - they own something the catalog lists as a substitute.

A missing base, modifier, mixer or sweetener kills the match. A missing garnish or ice only adds a note: nobody is short a drink because they have no cherries.

Misses are typed, never empty

Reason	What the UI says
filters	"Filters are too tight - N drinks match without them"
nearly	"You are one bottle away - add lime juice and 6 drinks unlock"
empty	"Nothing makes a full drink yet"

These fallbacks are the feature, not an edge case. Do not let a refactor turn them back into an empty state.

6. Traps that will cost you time

Each of these was a real bug. They are the main reason this document exists.

Several bottles deliberately share one catalog id

Eight gins all resolve sp-gin, so recipes match while the cabinet shows eight real bottles. Anything keyed off catalog id treats them as one. The Builder had exactly this bug - selecting one gin selected all of them - and now keys by inventory row id.

React Native Web ignores shadowRadius

Native shadow props render as a hard offset rectangle, not a blur. The shadow() helper in theme.ts emits a real boxShadow on web. Never add raw shadow* props to a style.

expo-sqlite cannot appear in the web bundle

Its native module has no web build and the static export fails. Persistence is split into store.ts and store.web.ts so Metro resolves by platform. A Platform.OS check inside one file does not work - the import alone breaks the build.

Never import from the @expo/vector-icons root

The index pulls every icon set and therefore every bundled font - about 4 MB of TTFs. Import @expo/vector-icons/build/createIconSet directly. The icon font itself is a 12 KB generated subset: after referencing a new glyph, run python scripts/build-icon-font.py.

Other sharp edges

- A glow must sit on an element that already has the border radius, or a pill's corners show as dark notches.
- Press puts its style on the Pressable, not an inner view - sizing must live on the element the parent lays out.
- The service worker uses cache: no-store on navigations; a plain fetch() served a stale app after deploys.
- Seed recipes merge by id, so id prefixes (seed-, book-, theme-) are stable. Renumbering breaks the merge.
- A recipe with no ingredients once matched vacuously and was served as pourable. Matching now requires one hard-role ingredient.

7. Decisions that look like mistakes but are not

Trending is not built. Descoped by the owner. src/api/trending.ts has the types settled and TRENDING_ENDPOINT = null; there is no tab. Do not treat it as unfinished work unless asked.

- **No Drizzle ORM** despite the spec calling for it - hand-written DDL with PRAGMA user_version migrations, currently schema v3.
- **No Reanimated and no Skia.** Skia was evaluated and rejected: on web it loads 2.9 MB of CanvasKit wasm, more than double the whole bundle.
- **Only the name is required** to save a recipe. A half-entered recipe at a party beats a form that will not close.
- **No drinking games or anything rewarding speed.** A scope decision. The age gate is blocking on first launch.
- **Themed drink names are evocative rather than lifted** from any franchise, because the app is publicly hosted.

8. Open work, roughly by value

1. **The cabinet is a wall of identical bottles.** Six gins render the same; only the name differs. The asset pipeline will not fix this - it dedupes the same way.
2. **No shopping list.** The generator already computes "add lime juice, unlocks 6 drinks", but nothing turns that into a list. Cheapest real feature win available.
3. **The editor can create a drink that can never be poured.** Correctly excluded from the generator, but it should say so.
4. **Guests land with an empty cabinet.** Storage is per-browser, so the 116-bottle bar is in the code but not the URL.
5. **The site is publicly indexable** behind a client-side-only age gate.
6. **@expo-google-fonts/nunito is an unused dependency** - the app moved to Outfit. Safe to remove.
7. **Trending**, per section 7.

9. Conventions

- TypeScript strict. npm run typecheck must pass before commit.

- Comments explain why, especially where something looks wrong but is not.
- Accessibility: touch targets at least 44px (chips use hitSlop), text at least 4.5:1 contrast, accessibility roles and labels on every control, reduced motion respected.
- No secrets in the client. The trending proxy exists precisely so a search key never ships in the bundle.

A fuller version of this document, including the complete file map, lives in the repo as `CONTEXT.md` - that is the copy a coding agent should read.